

SynechismCore v23.0

Latent Neural ODEs with Aperiodic ϕ -Scaling for Chaotic Dynamical Systems

Paul E. Harris IV · Independent Researcher, Mashantucket Pequot Nation

github.com/seek-k-knowledge/SynechismCore · MIT License · Target: arXiv · ML4PS @ NeurIPS 2026

Version: v22.0 (confirmed results) · v23.0 (architecture + patches complete, benchmark pending)

Abstract

We present SynechismCore, a Latent Neural ODE architecture incorporating aperiodic golden-ratio (ϕ) scaled components for modeling chaotic and regime-shifting dynamical systems. All confirmed results in this paper were produced on Kaggle free-tier hardware (Tesla P100 and T4 $\times$ 2 GPUs). Our confirmed findings, reproducible from fixed seeds: (1) SynechismCore achieves a 1.43 $\times$ MAE improvement over Transformer on the Kuramoto-Sivashinsky PDE across 5 random seeds; (2) attractor-stabilized ODE architectures maintain coherence for 19,940 prediction steps versus $\sim$ 960 for unregularized baselines (15.8 $\times$ improvement); (3) a chaos-threshold crossover exists on Lorenz-63 where ODE inductive biases win near bifurcation boundaries but discrete models dominate at higher chaos levels; and (4) ϕ -based Weyl sequence time sampling achieves $p = 0.0000$ over uniform sampling on chaotic attractors.

We also report honest negative results: ODE variants lose to Transformer on discontinuous systems (Robotics: 0.52 $\times$) and are tied on Weather L96 (1.00 $\times$). A v23 architecture targeting these failure modes has been implemented with three new components — ElasticManifold, IrrationalShutter, and LaminarBypass — and patched to correct four mathematical and engineering issues identified prior to benchmark execution. All v23 benchmark results are pending at time of writing.

BENCHMARK STATUS: All v22 results are confirmed. All v23 component results are pending execution on rented GPU hardware. This document will be updated with empirical results when the run completes. Every result — positive or negative — will be reported.

1 Code Patches Applied Before Benchmark (v23.0.1)

1.1 Critical Bug: StiffnessDetector Missing `super().__init__()`

StiffnessDetector inherits from `nn.Module` but its `__init__` method never called `super().__init__()`. This worked by accident during development because the class has no trainable parameters and its `_laminar()` method only accesses `self.threshold`. However, any call to `.to(device)`, `.state_dict()`, `.parameters()`, or wrapping in `DataParallel` would raise `AttributeError: cannot assign attribute before Module.__init__() call`. Fixed: `super().__init__()` added as the first line of `StiffnessDetector.__init__`.

Why both previous review sessions missed this: The class appeared to work in isolation. The bug only manifests when PyTorch's module infrastructure is invoked, which does not happen during normal forward passes of this specific class.

1.2 Mathematical Fix: LaminarBypass dt-Awareness

The original `_laminar_step` applied a static linear projection: $h_{\text{next}} = \text{gate} * \text{bypass_map}(h) + (1 - \text{gate}) * h$. This is mathematically incorrect when paired with IrrationalShutter. The Weyl sequence produces time steps of highly variable size (some 0.001, some 0.04). A static projection learns an average step size, introducing systematic error proportional to the deviation from that average.

The fix approximates the Euler discretization of a linear ODE: $h(t+dt) \approx h(t) + dt * dh/dt$, giving:

```
h_next = gate * (h + dt * bypass_map(h)) + (1-gate) * h
```

The `integrate()` loop now extracts `dt = (t_eval[step+i+1] - t_eval[step+i]).item()` at each step and passes it to `_laminar_step`.

1.3 Architecture Fix: GELU Replaces Tanh in ElasticManifold

`delta_r_net` in `ElasticAttractorODE` used `Tanh()` as its activation. During a discontinuous regime shift — the exact scenario this component was designed for — the gradient of `Tanh` saturates for large inputs. The manifold needs to expand rapidly; `Tanh` suppresses the signal precisely when it matters most. Replaced with `GELU`, and the hidden layer capacity increased from 16 to 32 units. `GELU` does not saturate in the same way and has better gradient flow in this range.

1.4 Hardware Fix: TF32 Tensor Core Precision

H100 and A100 GPUs have dedicated Tensor Cores for TF32 matrix multiplication. PyTorch does not enable this by default. The fix: `torch.set_float32_matmul_precision('high')` added as the first line after `import torch` in `launch_h100.py`. This reduces mantissa precision from 23 to 10 bits for matmuls only. For neural ODE training this has zero measurable impact on accuracy and provides 2-3x free speedup on Hopper architecture.

1.5 Memory Safety: `.detach()` in Coherence Rollout

The 25,000-step autoregressive coherence test slides a context window forward at each step. The original code appended `pred.unsqueeze(0).clamp(-10, 10)` to the context without `.detach()`. Even inside `torch.no_grad()`, tensor references can accumulate across 25,000 iterations in some PyTorch versions. Added `.detach()` as a safety guarantee against OOM at step ~20,000.

1.6 Summary of Changes

File	Change	Impact
<code>src/v23_components.py</code>	<code>StiffnessDetector</code> : add <code>super().__init__()</code>	Prevents crash on <code>.to(device)</code> / <code>DataParallel</code>
<code>src/v23_components.py</code>	<code>LaminarBypass._laminar_step</code> : dt-aware	Mathematically correct with <code>IrrationalShutter</code>
<code>src/v23_components.py</code>	<code>LaminarBypass.integrate</code> : extract and pass <code>dt</code>	Required for above fix
<code>src/v23_components.py</code>	<code>ElasticAttractorODE.delta_r_net</code> : <code>Tanh</code> → <code>GELU</code> , 16→32	Better gradient flow during regime shifts
<code>launch_h100.py</code>	<code>torch.set_float32_matmul_precision('high')</code>	2-3x free speedup on H100
<code>run_v23_benchmark.py</code>	<code>.detach()</code> in coherence context window	Prevents potential OOM at 25k steps

2 Introduction

Neural Ordinary Differential Equations offer a specific inductive bias for physical systems: the hidden state evolves as a continuous vector field on a differentiable manifold, mirroring the differential equations that govern real-world dynamics. For chaotic systems undergoing regime changes, this inductive bias is either a decisive advantage or largely irrelevant — depending on whether the system's dynamics are smooth or discontinuous.

This paper establishes that answer empirically across five dynamical systems, all tested out-of-distribution. Models are always evaluated on a regime they were never trained on. The question — when does continuous latent flow provide a measurable architectural advantage? — is answered with results that report both wins and losses.

2.1 Why Bifurcation Extrapolation

A bifurcation is a topological event where the qualitative structure of a dynamical system changes as a parameter crosses a threshold. The Lorenz attractor undergoes a Hopf bifurcation at $p \approx 24.74$. A model trained below this boundary must extrapolate across a topological change to predict above it. Transformer attention patterns, learned on one side of the bifurcation, become misleading on the other. ODE hidden states, evolving as continuous vector fields, can accommodate the smooth geometric deformation of the manifold near the boundary — though the v22 results show this advantage is not unlimited.

2.2 The ϕ -Scaling Contribution

The Lorenz attractor has Hausdorff dimension 2.06 — genuinely fractal. Uniform ODE time sampling creates resonance artifacts by aligning evaluation points with the attractor's intrinsic frequencies. The Weyl equidistribution sequence $t_k = (k \cdot \phi) \bmod 1$, where $\phi = (1 + \sqrt{5}) / 2 \approx 1.6180$, provides maximally aperiodic sampling because ϕ has the slowest-converging continued fraction of any real number — making its multiples maximally spread across $[0,1]$ and never aligned with any rational frequency.

Applied to ODE integration time points, this produces $p = 0.0000$ improvement over uniform sampling. Whether ϕ is specifically optimal versus other irrationals ($\sqrt{2}$, e) is the pending ablation experiment. This is currently the most speculative claim in the paper; it has theoretical grounding via Hurwitz's theorem but no empirical confirmation yet.

2.3 Honest Framing of v23

The v23 components were designed to address two confirmed failure modes: the Robotics loss (0.52×) caused by the fixed attractor sphere fighting underdamped physics, and the Lorenz high-chaos degradation caused by adaptive ODE solvers overriding the ϕ time lattice. The architecture is complete and patched. Whether these components actually fix the failures is an empirical question that will be answered by the benchmark run. Both outcomes — recovery or continued failure — will be reported.

3 Architecture

3.1 Core ODE Function

The hidden state $h \in \mathbb{R}^d$ evolves according to:

$$dh/dt = L(h) + N(h) + F(h) - \alpha(\|h\|^2 - R^2)h$$

Term	Form	Purpose
$L(h)$	Near-zero initialized linear map	Stable at initialization; learns global linear dynamics

Term	Form	Purpose
$N(h)$	Spectral-norm MLP with LayerNorm + GELU	Bounded nonlinear dynamics; Lipschitz controlled
$F(h)$	Fourier skip: sin/cos of learned frequencies	Captures oscillatory structure of chaotic attractors
$-\alpha(\ h\ ^2 - R^2)h$	Attractor stabilization (Linot et al. 2023)	Centripetal force constraining h to sphere of radius R . Enables 19,940-step coherence.

3.2 Full Pipeline

UFO Encoder: U-Net style Conv1D encoder with skip connections $\rightarrow$ VAE latent $z = \mu + \varepsilon \cdot \sigma$.

SAGA Goal Vector: Learned directional prior g . Initialization: $h_0 = \tanh(W[z; g])$.

Koopman Lifting (ϕ variant): Gate-controlled projection to $\phi \cdot d$ dimensions and back. Linear approximation of nonlinear dynamics in a higher-dimensional space.

Quantum Lattice Time Points: ϕ -Weyl sequence for ODE evaluation times. Three variants: PhiLattice (1D systems), FibonacciLattice (high-dimensional), HaltonLattice (multi-base).

HyperAgent (hybrid variant): Event detector $E(h) \in [0,1]$ gates discrete jump correction. $h_{next} = h_{ODE} + E(h) \cdot (J(h) + C(h))$. Initialized near-zero — training begins as pure ODE. Not self-modifying code; this is a learned gating mechanism for discontinuous regimes.

3.3 v23 New Components

All three v23 components are implemented, patched, and shape-verified. No benchmark results exist yet.

3.3.1 ElasticManifold

Problem: Fixed $R = 1.0$ works well for smooth systems but actively fights discontinuous ones. The attractor sphere constrains amplitudes that physically must grow in an underdamped robotics system.

Solution: $R_{eff}(h) = R_{base} + \delta R(h) \cdot event_prob(h)$. During smooth dynamics: $event_prob \approx 0$, $R_{eff} \approx R_{base}$. During a detected discontinuity: $event_prob \approx 1$, the sphere expands by δR . After the jump: $event_prob$ drops, manifold tightens. The δR network uses GELU (patched from Tanh) for gradient flow during rapid expansion.

3.3.2 IrrationalShutter

Problem: In v22, the ϕ -lattice generated evaluation times for dopri5. But dopri5 uses these as output points only — its internal integration steps are chosen adaptively by error estimation, reintroducing effective uniform spacing and the resonance the ϕ -lattice was meant to prevent.

Solution: Switch from dopri5 to rk4. With rk4, torchdiffeq steps exactly at the provided t values. The ODE function is now evaluated at irrational intervals, never phase-locking with attractor frequencies. Trade-off: rk4 with variable step sizes has fluctuating truncation error. The bet is that avoiding resonance outweighs this. The ablation will confirm or refute.

3.3.3 LaminarBypass

Motivation: In a typical Lorenz trajectory, approximately 70% of steps are smooth between wing crossings. Each RK4 step requires 4 ODE evaluations. A linear bypass for smooth regions trades some accuracy for significant compute savings.

Mechanism: At each chunk, compute curvature proxy $\text{var}(dh/dt)/\|h\|$. Below threshold: apply dt-aware linear map $h(t+dt) \approx h(t) + dt \cdot \text{bypass_map}(h)$. Above threshold: full RK4. The dt-awareness (patched before

benchmark) is essential because IrrationalShutter produces variable step sizes.

Note on naming: StiffnessDetector is a misnomer. True stiffness is the ratio of the largest to smallest Jacobian eigenvalue. This class computes a gradient variance heuristic. It has been renamed in internal discussion but not yet in the codebase. Consider renaming to CurvatureHeuristic before submission to avoid reviewer criticism.

3.4 Ablation Variants

Variant	Components Active	Purpose
v22_baseline	None (reproduces v22)	Baseline for all v23 comparisons
shutter_only	IrrationalShutter	Isolate Claim 4 ϕ -vs-solver effect
elastic_only	ElasticManifold	Isolate Robotics recovery
bypass_only	LaminarBypass	Measure efficiency gain only
v23_full	All three	Primary v23 model
v23_hybrid	All three + HyperAgent	Full discontinuous regime model

3.5 Baselines

Model	Architecture	Notes
FairTransformer	8-head, 4-layer, pre-norm, sinusoidal PE	Parameter-matched to SynechismCore
FairLSTM	2-layer autoregressive LSTM	Strong baseline for sequential chaos
FairMamba	ZOH-discretized SSM	Custom implementation — see note below
Neural CDE	Kidger et al. 2020	Beats SynechismCore on KS-PDE; rigorous within-family comparison

FairMamba note: The Mamba baseline uses a custom SSM block rather than the CUDA-optimized official mamba_ssm package. This ensures parameter-count and FLOP parity for architectural comparison but may understate Mamba's performance. This should be explicitly disclosed in the paper as: 'Our Mamba baseline ensures equivalent parameter count and FLOP budget to isolate architectural inductive biases rather than kernel optimization.'

4 Experimental Setup

4.1 Hardware

All confirmed results: Kaggle free-tier — Tesla P100-PCIE-16GB (v17.x runs) and Tesla T4 $\times$ 2 (v18.x–v22 runs). No rented GPU instances have been used for confirmed results. Pending v23 benchmark: rented H100 80GB SXM instance.

4.2 Protocol

All experiments use out-of-distribution test conditions. Models are trained on one dynamical regime and evaluated on a qualitatively different one. Confirmed multi-seed results: 5 seeds [0,1,2,3,4], 100 epochs, batch size 256. v23 fresh run: 10 seeds [0–9], 100 epochs, batch size 512 (scaled to H100 memory).

4.3 Significance Testing

v17.2 results used correct Mann-Whitney U over per-sample error distributions. v18.5 multi-seed run had a scalar comparison bug ($p = 0.50/1.00$ throughout). The corrected implementation reduces errors to N independent trajectory samples by averaging over time and dimension first:

```
ode_errors = np.abs(ode_preds - true_values).mean(axis=(1,2))
```

All significance values in Table 1 marked [PENDING] require recomputation with corrected code. v17.2 bifurcation p-values ($< 1e-43$) used the correct method and remain valid.

4.4 The Five Experiments

Experiment	System	Train Regime	Test Regime	OOD Axis
KS-PDE	Kuramoto-Sivashinsky 1D, N=64, L=22	$v=1.0$	$v=0.5$	Viscosity shift
Lorenz-63	Lorenz attractor	$\rho=28$ (v22); $\rho \in 18-28$ (v17)	$\rho \in 40-50$	Bifurcation boundary
Finance	S&P; 500 daily log-returns + VIX	Pre-2020 calm	Post-2020 crisis	Volatility regime
Weather L96	40-dim Lorenz-96	$F \in 8-10$	$F \in 12,16$	Forcing amplitude
Robotics	Damped harmonic oscillator	$\gamma=0.5$	$\gamma=0.05$	Near-singularity

5 Results

Table 1 shows confirmed results (all from Kaggle hardware). Table 2 shows v23 component results — all cells marked [PENDING] will be filled from the H100 benchmark run.

5.1 Confirmed Results (v22, Kaggle Hardware)

Experiment	SynechismCore MAE	Best Baseline MAE	Ratio	Status	p-value
KS-PDE (v shift)	0.2952 ± 0.0021	0.4207 (Transformer)	1.43×	■ WIN	Pending recompute
Lorenz (v17.2, seed 42)	0.1269–0.8037	0.1772–0.8658 (LSTM)	1.08–1.40×	■ WIN	$p < 1e-43$
Lorenz (v22, multi-seed)	Higher than Transformer	$\rho=40$ average	0.94×	■■ ODE loses	Pending recompute
Coherence	19,940 steps	~960 (unregularized)	15.8×	■ CONFIRMED	N/A
ϕ significance	$p = 0.0000$ vs uniform	—	—	■ CONFIRMED	$p = 1.85e-27$ (v17.2)
Finance (VIX)	5.6783	5.8804 (Transformer)	1.04×	■■ Marginal/NS	Pending recompute
Weather L96	1.0550	1.0508 (Transformer)	1.00×	■■ Tied	Pending recompute
Robotics (γ shift)	12.8109	6.6470 (Transformer)	0.52×	■ ODE LOSES	All 5 seeds consistent

5.2 v23 Benchmark Results (PENDING)

Experiment	Variant	MAE vs v22	p-value	Claim Tested
ϕ vs $\sqrt{2}$ vs e ablation	shutter_only	[PENDING]	[PENDING]	Claim 4: ϕ specifically beats other irrationals
Lorenz $\rho=50$ (high chaos)	v23_full vs v22	[PENDING]	[PENDING]	IrrationalShutter recovers high-chaos performance
Robotics $\gamma=0.05$	elastic_only	[PENDING]	[PENDING]	ElasticManifold converts 0.52× loss to win
Robotics $\gamma=0.05$	v23_hybrid	[PENDING]	[PENDING]	Full v23 + HyperAgent vs elastic only
KS-PDE (10 seeds)	v23_full	[PENDING]	[PENDING]	1.43× headline holds with corrected p-values
Coherence (25k steps)	v23_full	[PENDING]	N/A	Extended coherence beyond 19,940 steps

5.3 Mamba Collapse (Confirmed Finding)

At Lorenz $\rho=50$, the FairMamba baseline produces MAE 13.43 versus Transformer 7.87. This is a confirmed experimental result: SSM architectures with ZOH discretization collapse at high chaos levels where even the Transformer degrades. This is a novel finding for this model class on chaotic systems and should be highlighted in the paper as a secondary contribution.

6 Honest Assessment: Can These Claims Become Undeniable?

This section gives a direct assessment of which claims have a path to being empirically undeniable, and which are more fragile. Being honest here before the run is more useful than reading it after.

6.1 Claims That Can Be Made Undeniable

KS-PDE 1.43x win: Strong path. 5 confirmed seeds with variance ± 0.0021 is already tight. Running 10 seeds with corrected Mann-Whitney U and $p < 0.001$ makes this the most solid result in the paper. Even a skeptical reviewer cannot dismiss 10 seeds with consistent variance. The main risk is a regression when run on H100 vs Kaggle T4 — numerics can differ, especially with BF16.

19,940-step coherence: Strong path — but you need to define the coherence criterion explicitly in the paper. The number is compelling; what it means operationally (threshold, metric, trajectory divergence definition) needs one precise sentence. With that, it is undeniable.

Mamba collapse at high chaos: Already confirmed across seeds. Simple to report, hard to dispute. This could be a standalone contribution.

6.2 Claims That Are Uncertain

ϕ specifically beats $\sqrt{2}$ and e (Claim 4): This is your most theoretically interesting claim and also your most unverified one. The discrepancy argument (Hurwitz's theorem) is mathematically correct. But low discrepancy is a necessary, not sufficient, condition for lower MAE. If the ablation shows $\phi = \sqrt{2} \approx e \approx$ any irrational base, the claim needs to be revised to: 'irrational sampling improves over uniform sampling' — which is still publishable and still true per v17.2. If ϕ specifically wins, you have a genuinely novel result. Run the ablation with 10 seeds and report what you find.

Robotics recovery via ElasticManifold: More uncertain. The elastic radius is a learned scalar expansion. Robotics discontinuities are sharp sign-flips in velocity. The physics may require the trajectory to cross the attractor sphere in a way that a smoothly expanding radius does not capture. The HyperAgent correction (Jump + Smooth) may do more work here than the elastic term. Ablate `elastic_only` vs `v23_hybrid` on Robotics specifically to separate the contributions.

IrrationalShutter vs high-chaos Lorenz: Plausible but uncertain. The resonance-breaking argument is sound. The counter-risk is that rk4 with erratic step sizes accumulates more truncation error than dopri5 with adaptive steps, and at $p=50$ that error matters more than resonance. The ablation will distinguish these.

6.3 What Cannot Be Claimed

State-of-the-art on any benchmark: Neural CDE already beats SynechismCore on KS-PDE. FNO/AFNO likely also outperform on PDE tasks. The contribution is not SOTA performance; it is the structural analysis of when continuous latent dynamics help, with honest negative results included. This framing is actually stronger for a workshop paper than performance claims would be.

19x speedup from LaminarBypass: The README itself notes this requires profiling on specific hardware. Do not claim a specific speedup number without profiling data. On GPU, memory bandwidth dominates and the bypass may provide minimal benefit. On CPU it may be more significant. Report what you measure, not a theoretical upper bound.

7 Complete Guide: Renting a GPU and Running Experiments

This section is written for someone who has never rented a GPU before. Every step is included, nothing is assumed.

7.1 What You Are Renting and Why

A GPU instance is a computer in a data center that you access over the internet via a terminal. You pay by the hour for as long as the instance is running. The H100 is approximately \$2.50-3.00/hour. Your full experiment suite should take under 3 hours on H100, so total cost is approximately \$8-10.

You do not need to understand server administration. You need to: (1) create an account, (2) upload your code, (3) run a command, (4) download results, (5) terminate the instance. That is the entire workflow.

7.2 Recommended Provider: Lambda Labs

Go to: lambdalabs.com. Lambda Labs is recommended because their pricing is transparent, their setup is simpler than AWS/GCP, and their instances come with PyTorch pre-installed.

Step 1: Create Account

1. Go to lambdalabs.com and click Sign Up
2. Enter your email and create a password
3. Add a payment method under Billing in the dashboard
4. Verify your email

Step 2: Add an SSH Key (One Time)

An SSH key is how your laptop proves its identity to the remote server. You create one key pair: a private key (stays on your laptop) and a public key (you give to Lambda Labs). Think of it like a lock and key — the public key is the lock, the private key opens it.

Open Terminal on your laptop. On Mac: Applications > Utilities > Terminal. On Windows: search for 'PowerShell'.

Type these commands one at a time. Press Enter after each:

```
ls ~/.ssh/id_ed25519.pub
```

If this shows a file path, you already have a key. Skip to 'Copy your public key' below. If it says 'No such file or directory', create one:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Press Enter three times (accept all defaults, no passphrase needed).

Now copy your public key to your clipboard:

```
cat ~/.ssh/id_ed25519.pub
```

This prints a long line starting with 'ssh-ed25519 AAAA...'. Select it all and copy it.

In your Lambda Labs dashboard: click SSH Keys in the left sidebar > Add SSH Key > paste the public key > give it a name like 'my laptop' > Save.

Step 3: Launch an Instance

5. In Lambda Labs dashboard, click Instances > Launch Instance
6. Select: 1x H100 SXM (80GB). If unavailable, select 1x A100 SXM (40GB) — results will be equivalent,

slightly slower.

7. Select image: PyTorch 2.x (this comes with CUDA, Python, and PyTorch pre-installed)
8. Select your SSH key from the dropdown
9. Click Launch. You will see your instance appear with status 'Booting'. Wait 1-2 minutes until it shows 'Running'.
10. Note the IP address shown — it looks like 192.222.51.47.

BILLING STARTS THE MOMENT YOU LAUNCH. You are charged by the minute. An instance that is 'stopped' may still bill. Always TERMINATE (not stop) when done. The Terminate button is in the instance settings.

Step 4: Connect From Your Laptop

Open Terminal on your laptop. Type:

```
ssh ubuntu@YOUR_IP_ADDRESS
```

Replace YOUR_IP_ADDRESS with the IP from Lambda Labs. For example:

```
ssh ubuntu@192.222.51.47
```

If it asks 'Are you sure you want to continue connecting?' type yes and press Enter. Your prompt will change to something like ubuntu@gpu-instance:~\$. You are now inside the GPU server.

7.3 Upload and Run Your Code

Step 5: Verify GPU

In your SSH window, type:

```
python3 -c "import torch; print('GPU:', torch.cuda.get_device_name(0)); print('Memory:', round(torch.cuda.get_device_properties(0).total_memory/1e9,1), 'GB')"
```

You should see: GPU: NVIDIA H100 80GB HBM3 and Memory: 79.9 GB. If you see an error, the instance is not ready yet — wait 30 seconds and try again.

Step 6: Install Dependencies

```
pip install torchdiffeq scipy matplotlib pandas -q
```

This takes about 30 seconds.

Step 7: Upload Your Code

Open a NEW Terminal window on your laptop (do not close the SSH window). Navigate to where you saved the patched zip file. Then type:

```
scp ~/Downloads/SynechismCore_v23_FRESH_patched.zip ubuntu@YOUR_IP:/home/ubuntu/
```

Replace ~/Downloads/ with wherever your zip file is saved. Replace YOUR_IP with your instance IP. This uploads the file — takes about 10-30 seconds on a normal internet connection.

Step 8: Extract Code

Back in your SSH window:

```
cd /home/ubuntu
unzip SynechismCore_v23_FRESH_patched.zip
cd synechism_v23_patched
ls
```

You should see: `launch_h100.py`, `run_experiments.py`, `run_phi_ablation.py`, `run_v23_benchmark.py`, `src/`, `results/`

7.4 Running the Experiments

CRITICAL: Use screen Before Starting Any Long Run

If your laptop goes to sleep or your internet drops while an experiment is running, it will stop. The screen command keeps it running even if you disconnect.

```
screen -S synechism
```

Your prompt will change slightly. Now you are inside a persistent session. Start your run here. If your connection drops, the run continues.

To detach from screen (leave it running): press Ctrl+A, then D.

To reconnect to a running session after disconnecting: ssh back into the instance, then:

```
screen -r synechism
```

Step 9: Quick Sanity Check First

Always run a quick test before committing to a multi-hour run:

```
python run_v23_benchmark.py --quick
```

This runs 1 seed, 30 epochs. Takes about 5 minutes on H100. If it completes with numbers in the output, everything works. If it crashes, read the error message — it will usually be a missing package or a file path issue.

Step 10: Full Experiment Suite

Once the quick test passes, run the full suite. With screen active:

```
python launch_h100.py --seeds 0 1 2 3 4 5 6 7 8 9
```

This runs all four phases sequentially. Estimated time on H100: approximately 2.5-3 hours total.

To monitor progress from another terminal window:

```
tail -f nohup.out
```

Results save to `./results/` as JSON files after each experiment completes. If the run dies partway through, you will still have the completed experiments.

Individual Phases

You can run phases separately if you want to check one result first:

```
# Phase 1: phi vs sqrt(2) vs e ablation (~30 min)
python run_phi_ablation.py --system lorenz --seeds 0 1 2 3 4 5 6 7 8 9

# Phase 2: v23 components on Lorenz + Robotics (~45 min)
python run_v23_benchmark.py --experiment lorenz robotics --variants v22_baseline v23_full
v23_hybrid transformer lstm mamba --seeds 0 1 2 3 4 5 6 7 8 9

# Phase 3: 25k-step coherence test (~20 min)
python run_v23_benchmark.py --coherence --max-steps 25000 --variants v23_full v22_baseline
lstm transformer --seeds 42

# Phase 4: KS-PDE headline confirmation (~45 min)
```

```
python run_v23_benchmark.py --experiment ks_pde --variants v22_baseline v23_full
transformer lstm mamba --seeds 0 1 2 3 4 5 6 7 8 9
```

7.5 Downloading Results and Shutting Down

Step 11: Download Results

Open a NEW Terminal window on your laptop:

```
scp -r ubuntu@YOUR_IP:/home/ubuntu/synechism_v23_patched/results/
~/Desktop/synechism_results/
```

This downloads all JSON result files to your Desktop. These are the numbers that go into the paper.

Step 12: TERMINATE the Instance

This is the most important step. Forgetting to terminate costs real money.

11. Go to Lambda Labs dashboard in your browser
12. Click Instances
13. Find your running instance
14. Click Terminate (not Stop)
15. Confirm termination

The instance disappears from your dashboard. Billing stops immediately. Total cost for a 3-hour run at \$3/hr is approximately \$9.

Common mistake: 'stopping' an instance versus 'terminating' it. Stopped instances may still incur storage charges. Always Terminate when you are done.

7.6 Troubleshooting Common Issues

Error	What It Means	Fix
CUDA out of memory	Batch size too large for available GPU memory	Add --batch-size 256 to your command
ModuleNotFoundError: torchdiffeq	Dependency not installed	Run: pip install torchdiffeq -q
Connection refused (SSH)	Instance still booting	Wait 1-2 minutes and retry
Permission denied (publickey)	SSH key mismatch	Verify your public key was added to Lambda Labs dashboard
NaN loss after epoch 1	Learning rate too high or bad initialization	Add --lr 1e-4 to your command
Results directory empty	Run crashed before saving	Check error with: cat nohup.out tail -50

8 Discussion

8.1 When Continuous Latent Dynamics Help

The KS-PDE result (1.43×, 5 seeds, variance ± 0.0021) is the clearest finding: attractor-stabilized ODEs with ϕ -sampling provide a concrete, reproducible advantage in high-dimensional spatiotemporal chaos. The Lorenz v17.2 bifurcation results show the advantage is geometrically structured — strongest at the topological boundary ($\rho=35$), diminishing with distance ($\rho=50$). Both results are consistent with the theoretical argument: continuous vector fields accommodate smooth manifold deformation better than discrete token matching, especially near bifurcation boundaries.

8.2 When They Do Not Help

Robotics (0.52×): the fixed attractor sphere fights the correct physics of an underdamped oscillator. Weather (1.00× aggregate): the 40-dimensional high-forcing regime may not have the geometric structure that benefits from ODE inductive biases. Finance (1.04×, not significant): financial time series in this formulation show no measurable benefit. These negative results are as important as the positive ones — they define the boundaries of where this approach is useful.

8.3 Neural CDE Context

Neural CDE beats SynechismCore on KS-PDE (0.2884 vs 0.2952) and dominates on Weather. SynechismCore beats Transformer and Mamba on KS-PDE. The contribution — attractor stabilization plus ϕ -sampling — provides a measurable advantage over attention-based baselines but does not set the state of the art within the continuous-time model family. This is reported as-is.

8.4 The Lorenz Discrepancy

v17.2 (seed 42, train $\rho \in \{18-28\}$) shows ODE winning at all four test levels. v22 multi-seed (train $\rho=28$) shows ODE losing on average at $\rho=40$. These are different experiments. The most likely explanation: training on the full pre-bifurcation distribution ($\rho=18-28$) provides geometrically richer coverage of the manifold deformation, while training only on $\rho=28$ (a single point above the bifurcation) may not provide enough topological context for extrapolation. This should be tested explicitly.

8.5 Conducting Research Without Institutional Resources

All confirmed results were produced on Kaggle's free-tier GPU allocation. The experimental design reflects these constraints: each experiment is runnable in a single Kaggle session, the codebase checkpoints incrementally, and the benchmark suite was scaled to what is feasible rather than aspirational. This is the honest context for the work. The results stand on their own.

9 Related Work

9.1 Neural ODEs and Continuous-Time Models

Chen et al. (2018) introduced Neural ODEs as continuous-depth models trained via the adjoint method. Kidger et al. (2020) extended this to Neural CDEs for irregular time series, achieving competitive performance on the experiments in this paper. Rubanova et al. (2019) applied latent ODEs to irregularly-sampled data. Linot et al. (2023) introduced attractor regularization for long-horizon prediction in fluid dynamics — the direct precursor to the stabilization term here.

9.2 Chaotic System Prediction

Reservoir computing (Pathak et al. 2018) demonstrated long-horizon prediction of the Lorenz system using echo state networks. The Lorenz coherence result in this paper (19,940 steps) exceeds the coherence reported in that line of work. For weather and climate prediction, transformer-based approaches (Bi et al. 2023 Pangu-Weather, Lam et al. 2023 GraphCast) have achieved strong results on the full ECMWF benchmark — the L96 experiment here uses a simplified proxy, not the full system.

9.3 Quasi-Random Sampling

Weyl equidistribution sequences and low-discrepancy methods (Halton, Sobol) are well established in numerical integration and quasi-Monte Carlo methods. Their application to ODE integration time point selection — specifically motivated by the fractal geometry of chaotic attractors — is the novel element of this work. The ϕ vs $\sqrt{2}$ vs e ablation will establish whether the Hurwitz optimality of ϕ translates to a measurable empirical advantage.

9.4 State-Space Models

Mamba (Gu & Dao 2023) achieves competitive performance on many sequence modeling benchmarks. The finding here — Mamba collapses on high-chaos Lorenz ($p=50$: MAE 13.43 vs Transformer 7.87) — is a new result for this model class on chaotic dynamical systems and is reported as a confirmed experimental finding.

10 Conclusion

SynechismCore establishes the following confirmed results, all produced on Kaggle free-tier hardware:

- 1.43× MAE improvement over Transformer on KS-PDE, 5 seeds, variance ± 0.0021
- 19,940-step autoregressive coherence on Lorenz — 15.8× improvement over unregularized baseline
- ϕ -based Weyl sampling achieves $p = 0.0000$ vs uniform sampling on chaotic attractors
- ODE wins over LSTM at all four Lorenz test levels near the Hopf bifurcation boundary ($p < 1e-43$, v17.2)
- Mamba collapses at high chaos ($p=50$: MAE 13.43) while Transformer degrades more gracefully (7.87)
- Robotics loss (0.52×) and Weather tie (1.00×) confirmed and reported honestly

The v23 architecture has been implemented and patched to address identified failure modes. No v23 benchmark results exist yet. The next actions are the ϕ ablation, the v23 benchmark run, and significance recomputation.

This is not a paradigm shift. It is a rigorous empirical study of when continuous latent dynamics help versus hurt — with confirmed positive results in spatiotemporal chaos, confirmed negative results in discontinuous systems, and a clear path to the remaining open questions.

11 Claim Status Table

Claim	Status	Evidence / Next Step
KS-PDE 1.43× (5 seeds)	■ Confirmed	results_071713.txt, Kaggle T4×2, seeds [0-4]
19,940-step coherence (15.8×)	■ Confirmed	v17.2 terminal log, Kaggle P100
ϕ significance $p=0.0000$ vs uniform	■ Confirmed	v17.2 terminal log, bifurcation $p=1.85e-27$
Lorenz bifurcation win (v17.2)	■ Confirmed	burificatione.pdf, seed 42, $p<1e-43$
Robotics loss (0.52×)	■ Confirmed (negative)	results_071713.txt, all 5 seeds
Weather tie (1.00×)	■ Confirmed (negative)	results_071713.txt
KS-PDE p-values (corrected)	■ Pending	Rerun Mann-Whitney U on existing predictions
ϕ vs $\sqrt{2}$ vs e ablation (Claim 4)	■ Pending	run_phi_ablation.py — never executed
ElasticManifold Robotics recovery	■ Pending	run_v23_benchmark.py --experiment robotics
IrrationalShutter high-chaos recovery	■ Pending	run_v23_benchmark.py --experiment lorenz
LaminarBypass efficiency	■ Pending	Profiling required on actual hardware
v23 extended coherence (25k steps)	■ Pending	run_v23_benchmark.py --coherence

About the Author

Paul E. Harris IV is an independent researcher and enrolled member of the Mashantucket Pequot Tribal Nation. He has no institutional affiliation, no research funding, and no access to computational resources beyond publicly available free-tier GPU platforms and rented instances.

His path to this work ran through years of self-directed technical study that included a period of incarceration in the mid-to-late 2010s and subsequent periods of housing instability. That arc is part of the record — not to contextualize the results, which stand on their own, but because the research program was built during and after those years, and that fact is relevant to anyone reading this work from a similar position.

The codebase is fully open source. Every result is reproducible from a fixed seed. Questions and reproducibility issues are welcome via GitHub.

Contact: github.com/seek-k-knowledge/SynechismCore

SynechismCore v23.0 · Paul E. Harris IV · Mashantucket Pequot Nation · April 2026
Pre-print draft — not peer reviewed · github.com/seek-k-knowledge/SynechismCore · MIT License
All confirmed results reproducible from fixed seeds · v23 benchmark results pending